



## Carátula para entrega de prácticas

Facultad de Ingeniería

Laboratorios de docencia

# Laboratorio de Computación Salas A y B

*Profesor(a):* René Adrián Dávila Pérez

*Asignatura:* Programación Orientada a Objetos

*Grupo:* Grupo 7

*No de Práctica(s):* Estapa 1 "Proyecto"

*Integrante(s):* 323064039

426058744

120002850

323071378

323214636

426059576

*Semestre:* 2027 - 1

*Fecha de entrega:* 28 de Septiembre de 2026

*Observaciones:*

CALIFICACIÓN:

# Índice

|                          |           |
|--------------------------|-----------|
| <b>1. Introducción</b>   | <b>2</b>  |
| 1.1. Objetivos . . . . . | 2         |
| <b>2. Marco Teórico</b>  | <b>2</b>  |
| <b>3. Desarrollo</b>     | <b>3</b>  |
| <b>4. Resultados</b>     | <b>13</b> |
| <b>5. Conclusiones</b>   | <b>15</b> |

# 1. Introducción

La gestión operativa de un centro de salud demanda coordinar de forma rigurosa los flujos de asistencia, garantizando que cada paciente reciba atención de acuerdo con la especialidad médica pertinente y bajo restricciones estrictas de capacidad. En esta primera etapa del proyecto se aborda el diseño y construcción de un simulador de sistema hospitalario en Java orientado a modelar las interacciones fundamentales entre el personal médico, compuesto por médicos y enfermeros, y los pacientes que solicitan atención. El problema radica en administrar la asignación de pacientes respetando que cada médico pueda tener a su cargo hasta diez pacientes de su especialidad pero atender a consulta solo a uno a la vez, mientras que cada enfermero puede supervisar simultáneamente hasta tres pacientes hospitalizados de la misma especialidad. Asimismo, se requiere administrar el proceso clínico mediante transiciones de estado verificables y proveer mecanismos de ordenamiento alfabético para la inspección estructurada de los expedientes a cargo de cada profesional.

La motivación principal para sistematizar este entorno clínico reside en evitar saturaciones en el personal de salud y prevenir asignaciones erróneas que comprometan la calidad del servicio. En los entornos sanitarios reales, la sobreasignación de camas o la sobrecarga de consultas ambulatorias conducen a demoras y contingencias que pueden mitigarse mediante modelos computacionales que apoyen a la asignación de responsabilidades. La implementación de una solución orientada a objetos permite evaluar en un entorno controlado la viabilidad de restricciones hospitalarias.

## 1.1. Objetivos

- Implementar un simulador modular y robusto empleando el paradigma orientado a objetos, encapsulando los atributos y comportamientos en cuatro clases fundamentales: **Sistema**, **Medico**, **Enfermero** y **Paciente**, articuladas bajo una clase controladora **Main**.
- Modelar la sobrecarga de métodos para despachar solicitudes de asignación hacia médicos o enfermeros según la acción indicada.
- Administrar colecciones lineales dinámicas para el registro hospitalario.
- Controlar los estados del paciente mediante un tipo enumerado específico y materializar el ordenamiento alfabético por apellidos en las listas de pacientes vinculadas a cada uno de los profesionales de la salud.

# 2. Marco Teórico

El paradigma de Programación Orientada a Objetos constituye la base para representar entidades del mundo real mediante la abstracción de sus atributos de estado y sus métodos de comportamiento, protegiendo la coherencia de la información interna

mediante el encapsulamiento [2]. En el contexto hospitalario, entidades como médicos, enfermeros y pacientes poseen responsabilidades claramente delimitadas: mientras el médico diagnostica y prescribe tratamientos, el personal de enfermería ejecuta las pautas terapéuticas y mantiene la supervisión continua del paciente internado [3].

La sobrecarga permite definir múltiples métodos con un identificador idéntico dentro de una misma clase, diferenciándose exclusivamente por su lista de parámetros formales en número, orden o tipo de dato. Esta técnica resulta idónea para centralizar operaciones semánticamente análogas pero estructuralmente distintas en la clase principal del hospital; por ejemplo, la asignación de un paciente a un médico requiere evaluar turnos y especialidad ambulatoria, mientras que la canalización hacia un enfermero exige vincular al paciente, médico remitente, especialidad y prescripción farmacológica específica [5].

Un paciente no puede recibir tratamiento sin haber sido evaluado previamente en consulta, ni puede ingresar a consulta sin hallarse registrado formalmente en lista de espera. La definición de un tipo enumerado en Java proporciona seguridad de tipos en tiempo de compilación, restringiendo los estados admisibles a valores canónicos discretos y evitando inconsistencias derivadas de cadenas de texto arbitrarias.

Por último, la administración de registros en memoria recurre a las estructuras dinámicas provistas por el marco de colecciones de Java, en particular listas enlazadas o arreglos dinámicos y mapas ordenados [5]. Las listas lineales admiten adiciones continuas y acceso posicional indexado, mientras que los mapas ordenados garantizan unicidad e indexación de catálogos como el de especialidades médicas. El ordenamiento alfabético de los elementos se consigue aplicando expresiones lambda sobre comparadores de cadenas, garantizando la presentación sistemática por apellido de los pacientes atendidos.

### 3. Desarrollo

Como parte de la fase exploratoria del proyecto, se formuló una consulta técnica común orientada a contrastar enfoques sobre la modelación de entidades clínicas, sobrecarga de métodos y transiciones de estado en Java, el modelado orientado a objetos de interacciones hospitalarias y sobrecarga de métodos.

La síntesis de las consultas confirma que una arquitectura orientada a objetos desacoplada exige que el sistema central actúe como coordinador neutral de recursos, delegando la contabilidad interna de pacientes a las instancias médicas y de enfermería, la comparación analítica detallada de las respuestas queda programada para su incorporación en la siguiente iteración del reporte una vez concluidas las pruebas del simulador.

La solución implementada se estructura en cinco clases autónomas, la primera de ellas la clase `Main` implementa la interfaz interactiva por consola basada en un ciclo continuo con menú numérico que permite cargar escenarios de demostración, registrar entidades individuales, despachar consultas y tratamientos, e inspeccionar los catálogos en memoria, de esta manera la Figura 3.1 ilustra el flujo de control general de la clase `Main`.

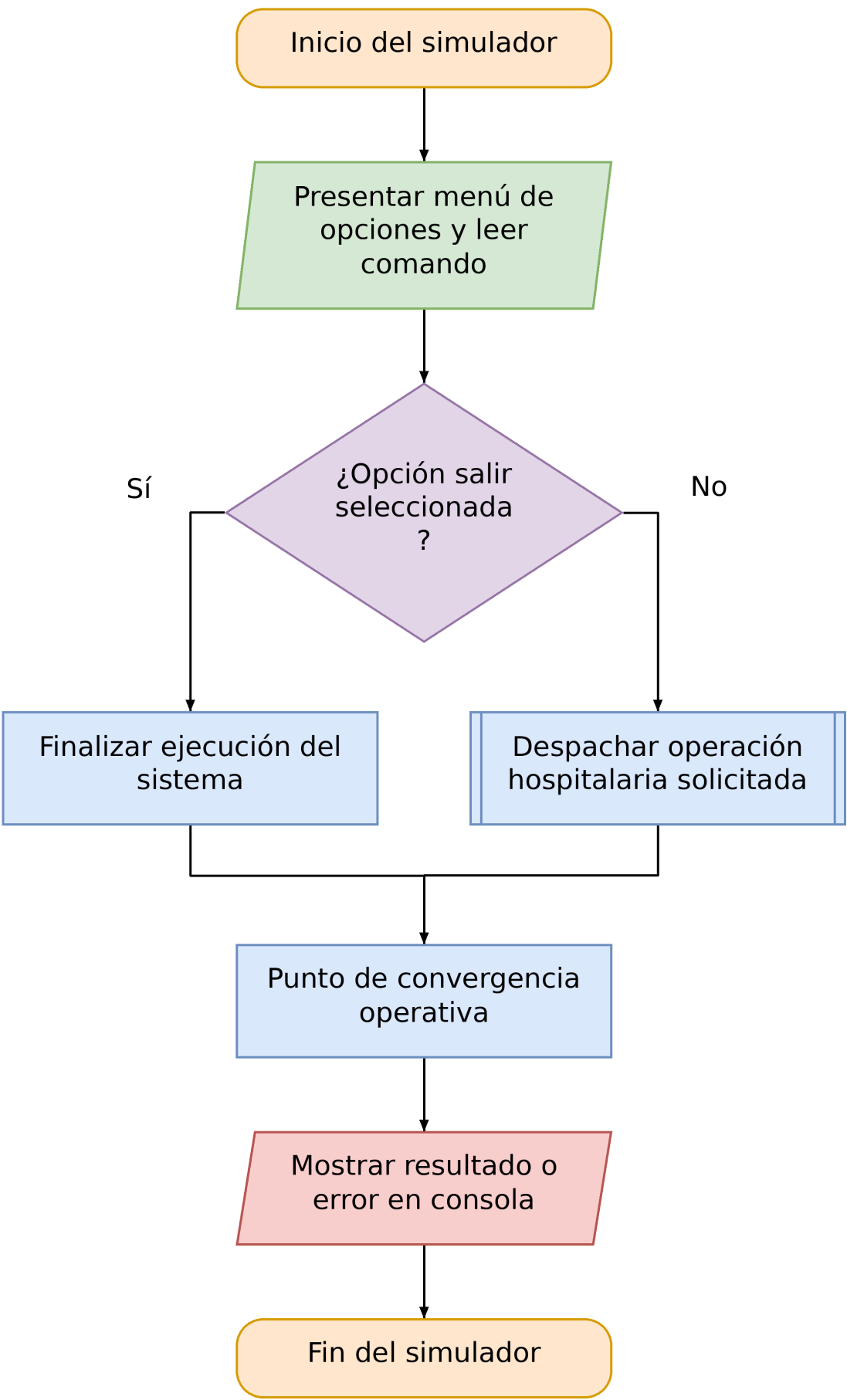
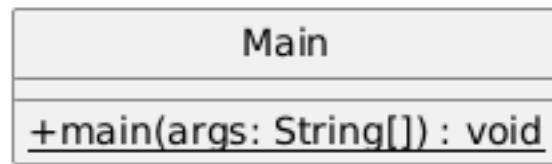


Figura 3.1: Flujo de la clase `Main`

La estructura de la clase `Main` se describe en la Figura 3.2.



**Figura 3.2:** Estructura de la clase `Main`

Pseudocodigo de la clase `Main`:

Clase `Main`

```
Función Principal():
    hospital = Nulo
    opcion = -1
    Mientras opcion != 0 Hacer:
        Mostrar menú interactivo (Opciones 0 al 13)
        Leer opcion
        Segun opcion Hacer:
            Caso 1:
                Cargar datos demo (3 especialidades,
                3 médicos, 3 enfermeros, 3 pacientes)
            Caso 2:
                hospital = Nuevo Sistema(leerNombre())
            Caso 3..6:
                Validar que exista hospital y
                especialidades registradas
                Instanciar Medico / Enfermero / Paciente
                y dar de alta con registroEnSistema()
            Caso 7:
                Seleccionar médico disponible ->
                medico.solicitarPaciente(hospital)
            Caso 8:
                Seleccionar paciente en espera ->
                paciente.solicitarConsulta()
            Caso 9:
                Seleccionar médico con paciente activo ->
                medico.darTratamiento(paciente,
                leerPrescripcion())
            Caso 10..13:
                Desplegar listados generales de Especialidades,
                Médicos, Enfermeros y Pacientes invocando
                verListaPacientes() en orden alfabético
            Caso 0:
                Imprimir "Saliendo del sistema..."
        FinSegun
    FinMientras
```

La clase `Sistema` representa el núcleo administrativo del hospital, manteniendo colecciones dinámicas para médicos, enfermeros y pacientes, así como un mapa ordenado para el registro de especialidades, se incorpora el método sobrecargado `asignarPaciente`: su primera versión localiza un paciente en espera de la especialidad correspondiente al médico solicitante; su segunda versión recibe la prescripción médica y vincula al paciente con un enfermero de la misma especialidad que posea menos de tres pacientes a cargo, ilustrado en la Figura 3.3 la cual detalla el flujo de asignación y registro de la clase `Sistema`.

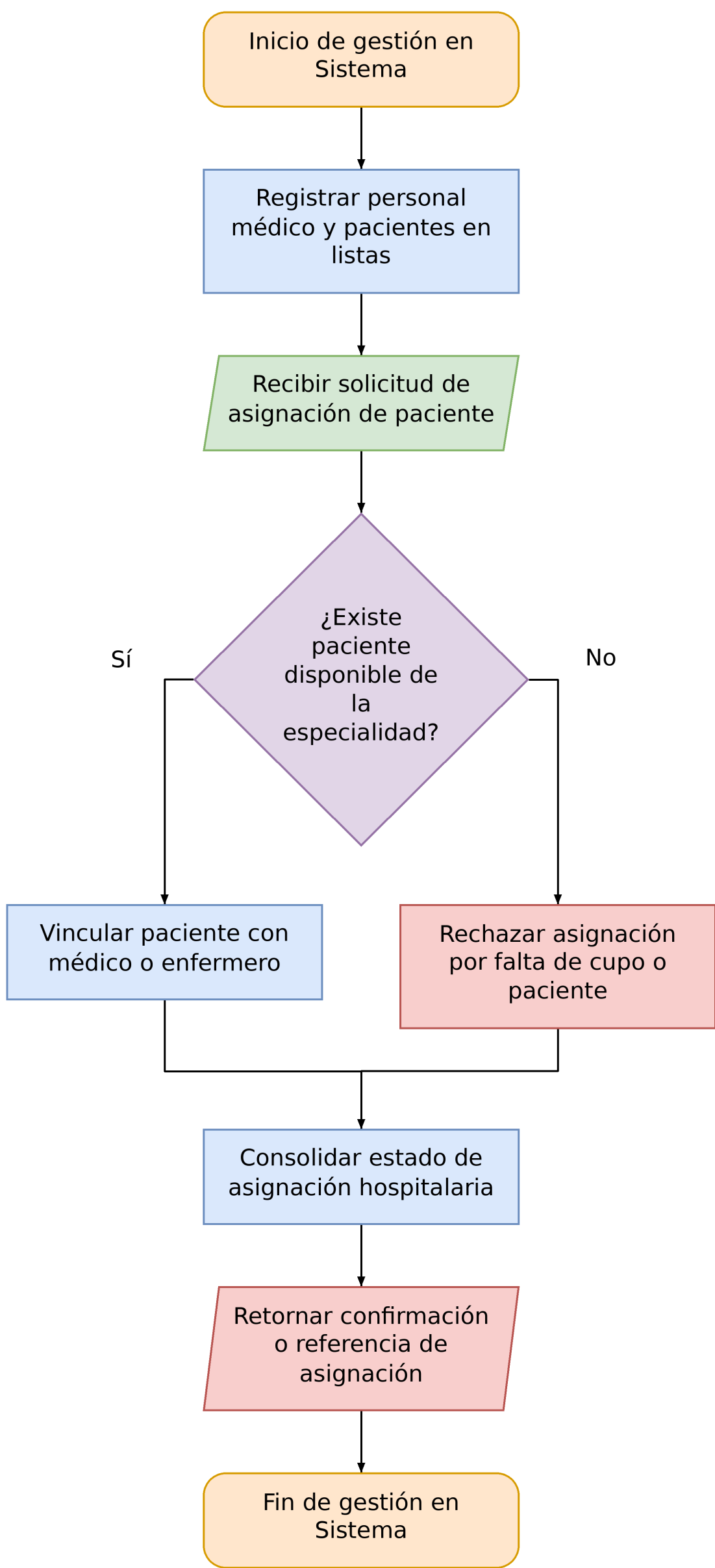


Figura 3.3: Flujo de la clase Sistema

La especificación de atributos y métodos de la clase **Sistema** se presenta en la Figura 3.4.

| Sistema                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ~nombreHospital : String<br>~noMedicos : int<br>~noEnfermeros : int<br>~noPacientes : int<br>~m : ArrayList<Medico><br>~e : ArrayList<Enfermero><br>~p : ArrayList<Paciente><br>~spec : TreeMap<Integer,String>                                                                                                                                                                                                                                                                                                                                                                                                                          |
| +Sistema(nombreHospital: String)<br>+registroMedico(med: Medico) : void<br>+registroEnfermero(enf: Enfermero) : void<br>+registroPaciente(pac: Paciente, especialidad: int) : boolean<br>+asignarPaciente(medico: Medico, especialidad: int) : Paciente<br>+asignarPaciente(paciente: Paciente, medico: Medico, especialidad: int, prescripcion: String) : boolean<br>+setEspecialidad(nombre: String) : void<br>+getEspecialidad() : void<br>+getNumeroEspecialidades() : int<br>+getNombreHospital() : String<br>+getMedicos() : ArrayList<Medico><br>+getEnfermeros() : ArrayList<Enfermero><br>+getPacientes() : ArrayList<Paciente> |

**Figura 3.4:** Estructura de la clase Sistema

Pseudocodigo de la clase Sistema:

Clase Sistema

Atributos:

```

nombreHospital: Cadena
noMedicos, noEnfermeros, noPacientes: Entero
medicos: Lista de Medico
enfermeros: Lista de Enfermero
pacientes: Lista de Paciente
especialidades: Diccionario Ordenado (ID -> Nombre)

```

Constructor(nombreHospital):

```

Inicializar contadores en 0 y colecciones vacías

```

Método registroMedico(med):

```

Agregar med a medicos; noMedicos++

```

Método registroEnfermero(enf):

```

Agregar enf a enfermeros; noEnfermeros++

```

Método registroPaciente(pac, esp) -> Booleano:

```

Para cada med En medicos Hacer:
    Si med.especialidad == esp Y
        med.noPacientes < 10 Entonces:
            Agregar pac a pacientes; noPacientes++

```

```

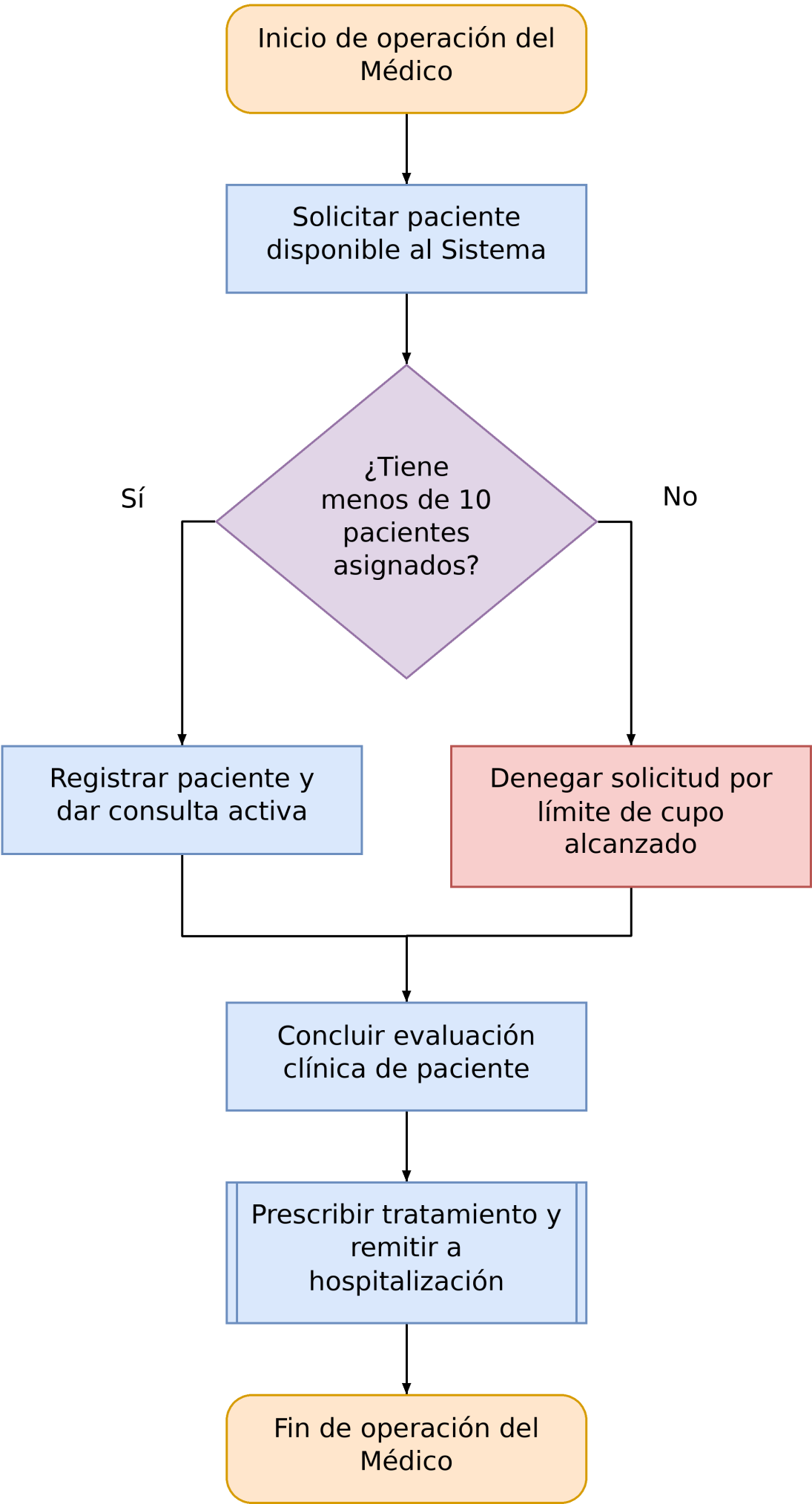
        Retornar Verdadero
    FinSi
FinPara
Retornar Falso

// Sobrecarga 1: Búsqueda de paciente para médico
Método asignarPaciente(medico, esp) -> Paciente:
    Para cada pac En pacientes Hacer:
        Si pac.especialidad == esp Y
            pac.estado == EN_ESPERA Y
            pac.medico == Nulo Entonces:
                Retornar pac
        FinSi
    FinPara
Retornar Nulo

// Sobrecarga 2: Derivación a enfermero
Método asignarPaciente(paciente, medico, esp,
                        prescripcion) -> Booleano:
    Para cada enf En enfermeros Hacer:
        Si paciente.estado == EN_CONSULTA Y
            enf.puedeAtender(esp) Entonces:
                enf.darTratamiento(paciente, prescripcion)
                Retornar Verdadero
        FinSi
    FinPara
Retornar Falso
FinClase
```

La clase `Medico` encapsula el nombre, cédula profesional, especialidad y el contador de pacientes asignados, posee como condición que su colección no puede superar diez pacientes y que su atributo de atención activa solo admite una referencia a la vez durante la consulta.

La Figura 3.5 muestra el diagrama de flujo correspondiente al ciclo de atención de la clase `Medico`.



**Figura 3.5:** Flujo de la clase `Medico`

La estructura estática de la clase `Medico` se sintetiza en la Figura 3.6.

| Medico                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ~nombre : String<br>~cedula : int<br>~especialidad : int<br>~noPacientes : int<br>~atendiendo : Paciente<br>~sis : Sistema<br>~p : ArrayList<Paciente>                                                                                                                                                                                                                                                                                                                                                                                                          |
| +Medico(nombre: String, cedula: int, especialidad: int)<br>+Medico(nombre: String, cedula: int, especialidad: int, sis: Sistema)<br>+registroEnSistema(s: Sistema) : void<br>+solicitarPaciente(s: Sistema) : boolean<br>+darConsulta(paciente: Paciente) : boolean<br>+darTratamiento(paciente: Paciente, tratamiento: String) : boolean<br>+verListaPacientes() : void<br>+getEspecialidad() : int<br>+getNoPacientes() : int<br>+getNombre() : String<br>+getCedula() : int<br>+getAtendiendo() : Paciente<br>+getPacientesAsignados() : ArrayList<Paciente> |

**Figura 3.6:** Estructura de la clase `Medico`

Pseudocodigo de la clase `Medico`:

Clase `Medico`

Atributos:

nombre: Cadena  
cedula, especialidad, noPacientes: Entero  
atendiendo: Paciente (Nulo)  
sis: Sistema (Nulo)  
pacientesAsignados: Lista de Paciente

Constructor(nombre, cedula, especialidad):

Asignar atributos iniciales con noPacientes = 0

Método `registroEnSistema(sistema)`:

sis = sistema  
sistema.registroMedico(estesteMedico)

Método `solicitarPaciente(sistema) -> Booleano`:

Si noPacientes < 10 Entonces:

```
p = sistema.asignarPaciente(esteMedico, especialidad)
Si p != Nulo Entonces:
    Agregar p a pacientesAsignados
    p.setMedico(esteMedico)
    noPacientes = noPacientes + 1
    Retornar Verdadero
FinSi
FinSi
Retornar Falso

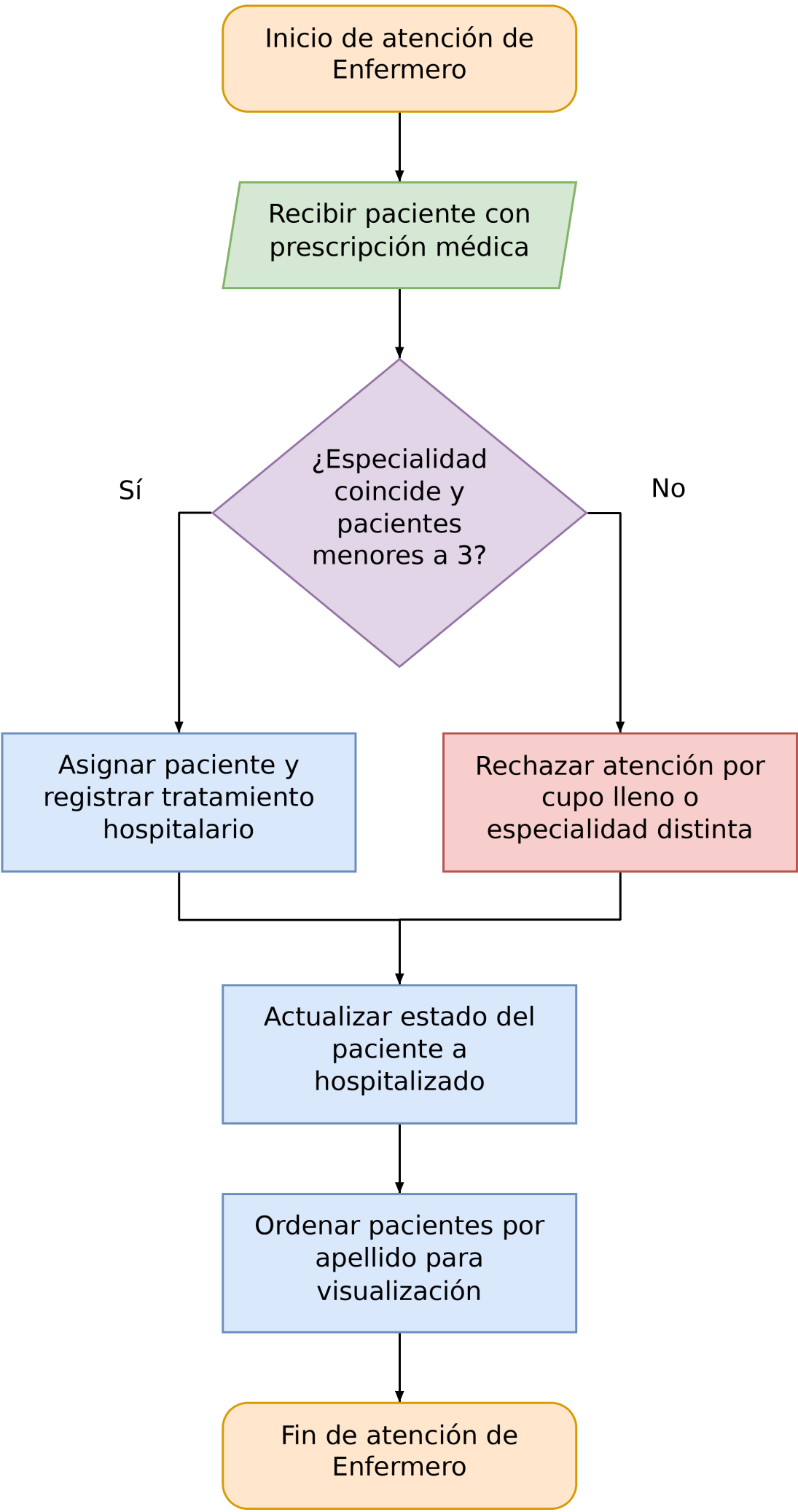
Método darConsulta(paciente) -> Booleano:
Si paciente está en pacientesAsignados Y
    atendiendo == Nulo Entonces:
        atendiendo = paciente
        paciente.setEstado(EN_CONSULTA)
        Retornar Verdadero
FinSi
Retornar Falso

Método darTratamiento(paciente, prescripcion) -> Booleano:
Si paciente == atendiendo Y
    sis.asignarPaciente(paciente, esteMedico,
                        especialidad, prescripcion) Entonces:
        atendiendo = Nulo
        paciente.setTratamiento(prescripcion)
        paciente.setEstado(EN_HOSPITAL)
        Retornar Verdadero
FinSi
Retornar Falso

Método verListaPacientes():
    Ordenar pacientesAsignados por apellido de forma A-Z
    Para cada p En pacientesAsignados Hacer:
        Imprimir p.apellido + " " + p.nombre +
            " - Especialidad: " + p.especialidad
FinClase
```

La clase `Enfermero` gestiona el cuidado directo de pacientes hospitalizados, validando que el paciente pertenezca a su misma especialidad y que el total de internados bajo su supervisión sea estrictamente menor a tres, entre sus metodos el encargado de `darTratamiento` actualiza el estado del paciente a hospitalizado y almacena la prescripción médica.

La Figura 3.7 esquematiza el flujo operativo de la clase `Enfermero`.



**Figura 3.7:** Flujo de la clase `Enfermero`

La estructura de componentes de la clase **Enfermero** se ilustra en la Figura 3.8.

| Enfermero                                                                                                                                                                                                                                                                                                                                                    |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ~nombre : String<br>~cedula : int<br>~especialidad : int<br>~noPacientes : int<br>~p : ArrayList<Paciente><br>~sis : Sistema                                                                                                                                                                                                                                 |
| +Enfermero(nombre: String, cedula: int, especialidad: int)<br>+registroEnSistema(s: Sistema) : void<br>+verListaPacientes() : void<br>+darTratamiento(paciente: Paciente, tratamiento: String) : boolean<br>+puedeAtender(especialidad: int) : boolean<br>+getNombre() : String<br>+getCedula() : int<br>+getEspecialidad() : int<br>+getNoPacientes() : int |

**Figura 3.8:** Estructura de la clase Enfermero

Pseudocodigo de la clase Enfermero:

Clase Enfermero

Atributos:

nombre: Cadena  
cedula, especialidad, noPacientes: Entero  
pacientesACargo: Lista de Paciente  
sis: Sistema (Nulo)

Constructor(nombre, cedula, especialidad):

Asignar atributos iniciales con noPacientes = 0

Método registroEnSistema(sistema):

sis = sistema  
sistema.registroEnfermero(estenfermero)

Método puedeAtender(espRequerida) -> Booleano:

Retornar (noPacientes < 3 Y  
especialidad == espRequerida)

Método darTratamiento(paciente, prescripcion) -> Booleano:

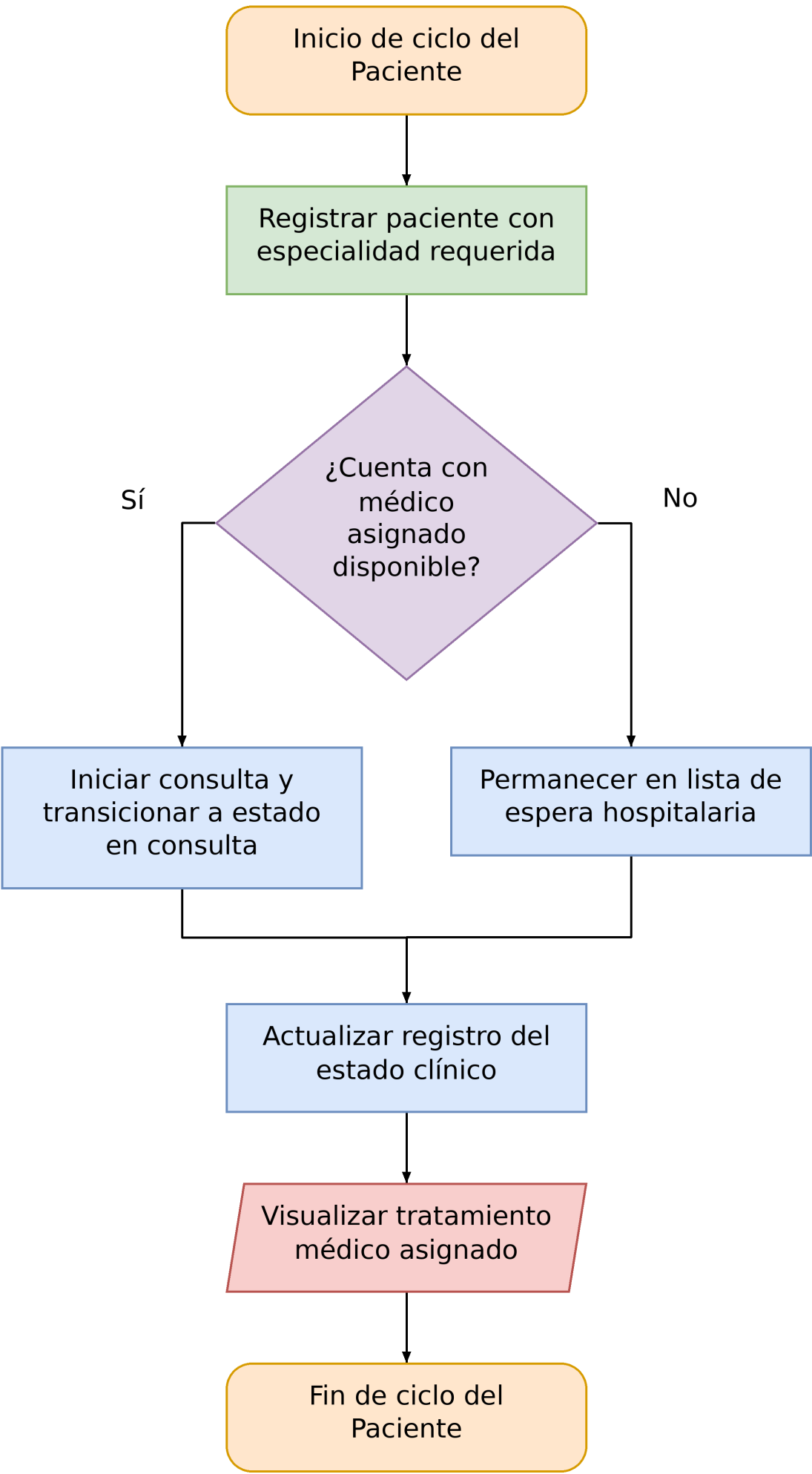
Si paciente está en pacientesACargo 0  
noPacientes >= 3 Entonces:  
Retornar Falso

```
FinSi
Agregar paciente a pacientesACargo
paciente.setEnfermero(esteEnfermero)
paciente.setTratamiento(prescripcion)
paciente.setEstado(EN_HOSPITAL)
noPacientes = noPacientes + 1
Retornar Verdadero

Método verListaPacientes():
  Ordenar pacientesACargo por apellido de forma A-Z
  Para cada p En pacientesACargo Hacer:
    Imprimir p.apellido + " " + p.nombre +
      " - Especialidad: " + p.especialidad
FinClase
```

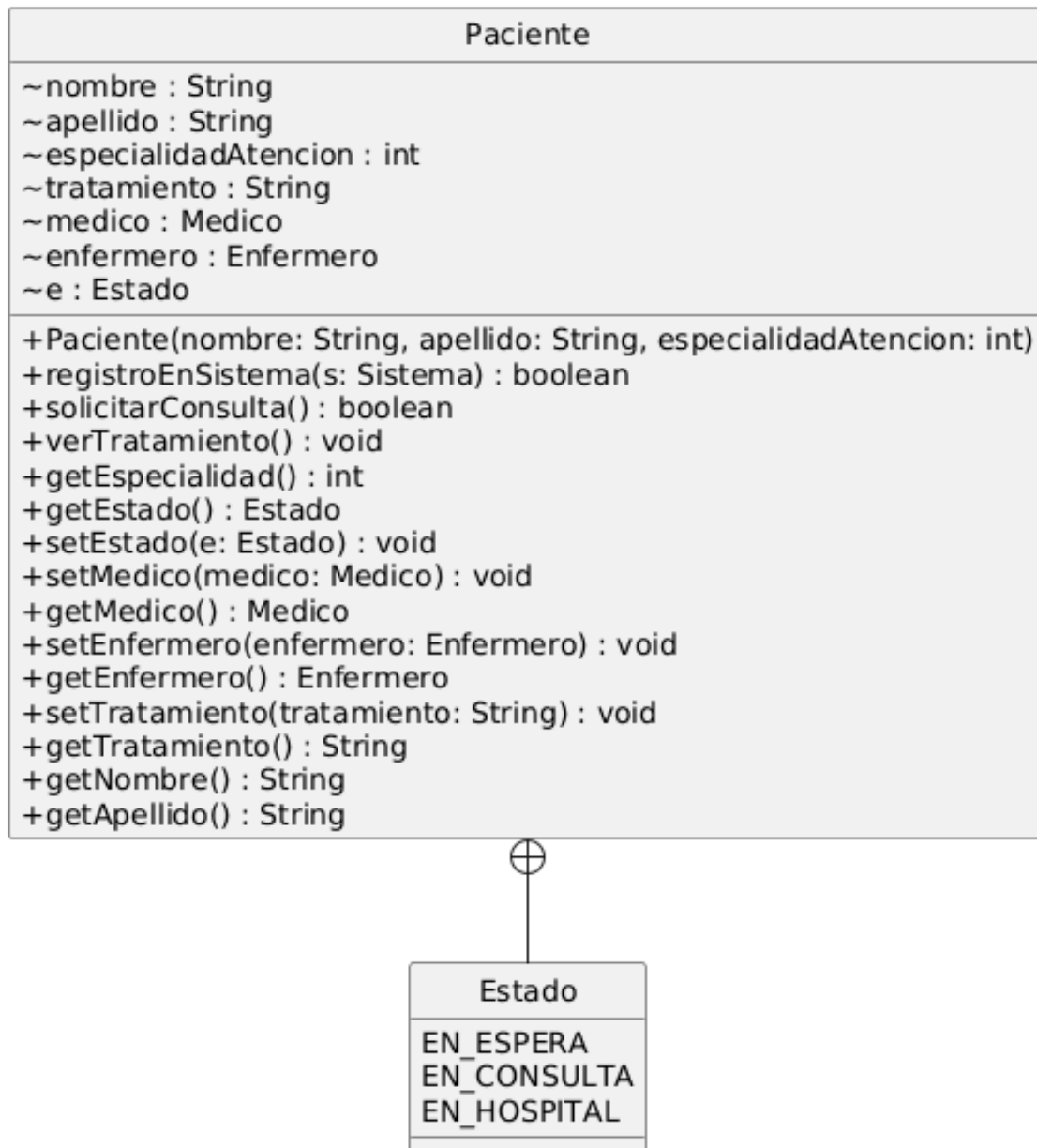
La clase `Paciente` almacena la filiación, especialidad requerida, tratamiento asignado, referencias a sus profesionales asignados y su estado clínico modelado mediante el enumerado `Estado` (con los valores en espera, en consulta y hospitalizado).

La Figura 3.9 expone el ciclo de vida de la entidad `Paciente`.



**Figura 3.9:** Flujo de la clase `Paciente`

La composición interna de la clase `Paciente` se expone en la Figura 3.10.



**Figura 3.10:** Estructura de la clase `Paciente`

Pseudocódigo de la clase `Paciente`:

Clase `Paciente`

Atributos:

```

nombre, apellido: Cadena
especialidadAtencion: Entero
tratamiento: Cadena ("Sin tratamiento")
medico: Medico (Nulo)
  
```

```

    enfermero: Enfermero (Nulo)
    estado: Estado (EN_ESPERA)
        // Enum: EN_ESPERA, EN_CONSULTA, EN_HOSPITAL

Constructor(nombre, apellido, especialidad):
    Asignar atributos iniciales

Método registroEnSistema(sistema) -> Booleano:
    Retornar sistema.registroPaciente(estePaciente,
                                        especialidadAtencion)

Método solicitarConsulta() -> Booleano:
    Si medico != Nulo Y
        medico.darConsulta(estePaciente) Entonces:
            estado = EN_CONSULTA
            Retornar Verdadero
    FinSi
    Retornar Falso

Método verTratamiento():
    Imprimir tratamiento
FinClase

```

El requerimiento de visualización ordenada se resuelve implementando en **Medico** y **Enfermero** el método **verListaPacientes**, el cual ejecuta una ordenación lexicográfica basada en **compareToIgnoreCase** sobre el apellido del paciente previo a desplegar los datos en consola, garantizando la presentación en formato de apellido, nombre y especialidad.

## 4. Resultados

Para validar exhaustivamente la lógica del sistema hospitalario y la verificación de invariantes, se diseñó una versión precargada a través del menú de consola, esta prueba inicia cargando los datos de demostración preconfigurados, los cuales registran tres especialidades (Cardiología, Pediatría y Traumatología), tres médicos, tres enfermeros y tres pacientes en espera.

Posteriormente, se solicita la asignación de dos pacientes consecutivos de la especialidad de Cardiología hacia el médico especialista correspondiente (Dr. Carlos López). El sistema asigna sucesivamente a Gabriel Soriano Méndez y a Ledesma García Sánchez, incrementando la carga del médico a dos pacientes.

A continuación, el paciente Soriano Méndez solicita consulta clínica formal, y el sistema valida la disponibilidad del médico, transicionando al paciente al estado en consulta activa, como acto seguido, el médico emite la prescripción médica e instruye el tratamiento; el sistema activa la sobrecarga de asignación hacia enfermería, canalizando al paciente con la enfermera Ana Sánchez e internándolo bajo el estado hospitalizado.

Finalmente, se invocan las opciones de inspección de personal médico y de enfermería. La visualización de médicos comprueba que el Dr. López lista a sus pacientes ordenados alfabéticamente por apellido (García Sánchez antes de Soriano Méndez), y la inspección de enfermeros confirma que la enfermera Sánchez tiene a su cargo al paciente hospitalizado. La Figura 4.1 presenta la evidencia de ejecución completa capturada en la terminal.

```
=====
                        SISTEMA HOSPITALARIO
=====
[1]  Cargar datos demo (3 de cada tipo)
-----
[2]  Agregar hospital
[3]  Agregar especialidad
[4]  Agregar medico
[5]  Agregar enfermero
[6]  Agregar paciente
-----
[7]  Medico solicita paciente
[8]  Paciente solicita consulta
[9]  Medico da tratamiento
-----
[10] Ver especialidades
[11] Ver medicos
[12] Ver enfermeros
[13] Ver pacientes
-----
[0]  Salir
=====
Selecciona una opcion: 1
Objetos demo cargados en: Hospital Central Universitario
- 3 Especialidades registradas
- 3 Medicos registrados
- 3 Enfermeros registrados
- 3 Pacientes en espera

=====
                        SISTEMA HOSPITALARIO
=====
[1]  Cargar datos demo (3 de cada tipo)
-----
[2]  Agregar hospital
[3]  Agregar especialidad
[4]  Agregar medico
[5]  Agregar enfermero
[6]  Agregar paciente
-----
[7]  Medico solicita paciente
[8]  Paciente solicita consulta
[9]  Medico da tratamiento
-----
[10] Ver especialidades
[11] Ver medicos
[12] Ver enfermeros
[13] Ver pacientes
-----
[0]  Salir
=====
Selecciona una opcion: 7

Selecciona el medico que solicita paciente:
[1] Dr. Carlos Lopez (Esp: 1, Pacientes: 0)
[2] Dra. Laura Gomez (Esp: 2, Pacientes: 0)
[3] Dr. Juan Ramirez (Esp: 3, Pacientes: 0)
Numero: 1
Paciente asignado con exito.

=====
                        SISTEMA HOSPITALARIO
=====
[1]  Cargar datos demo (3 de cada tipo)
-----
[2]  Agregar hospital
[3]  Agregar especialidad
[4]  Agregar medico
[5]  Agregar enfermero
[6]  Agregar paciente
-----
[7]  Medico solicita paciente
[8]  Paciente solicita consulta
[9]  Medico da tratamiento
-----
[10] Ver especialidades
[11] Ver medicos
[12] Ver enfermeros
[13] Ver pacientes
-----
[0]  Salir
=====
Selecciona una opcion: 8

Selecciona el paciente que entrara a consulta:
[1] Soriano Mendez Gabriel - Estado: EN_ESPERA
[2] Garcia Sanchez Ledesma - Estado: EN_ESPERA
[3] Zapata Moran Hugo - Estado: EN_ESPERA
Numero: 1
Consulta iniciada.
Estado actual de Soriano Mendez: EN_CONSULTA

=====
                        SISTEMA HOSPITALARIO
=====
[1]  Cargar datos demo (3 de cada tipo)
-----
[2]  Agregar hospital
[3]  Agregar especialidad
[4]  Agregar medico
[5]  Agregar enfermero
[6]  Agregar paciente
-----
[7]  Medico solicita paciente
[8]  Paciente solicita consulta
[9]  Medico da tratamiento
-----
[10] Ver especialidades
[11] Ver medicos
[12] Ver enfermeros
[13] Ver pacientes
-----
[0]  Salir
=====
Selecciona una opcion: 9

Selecciona el medico que dara el tratamiento:
[1] Dr. Carlos Lopez
[2] Dra. Laura Gomez
[3] Dr. Juan Ramirez
Numero: 1
Atendiendo a: Soriano Mendez Gabriel
Prescripcion medica: Reposo y amiodarona
Tratamiento asignado y paciente hospitalizado.
Estado del paciente: EN_HOSPITAL

=====
                        SISTEMA HOSPITALARIO
=====
[1]  Cargar datos demo (3 de cada tipo)
-----
[2]  Agregar hospital
[3]  Agregar especialidad
[4]  Agregar medico
[5]  Agregar enfermero
[6]  Agregar paciente
-----
[7]  Medico solicita paciente
[8]  Paciente solicita consulta
[9]  Medico da tratamiento
-----
[10] Ver especialidades
[11] Ver medicos
[12] Ver enfermeros
[13] Ver pacientes
-----
[0]  Salir
=====
Selecciona una opcion: 0
Saliendo del sistema...
```

Figura 4.1: Ejecución y pruebas del simulador hospitalario en consola

La salida obtenida demuestra que las restricciones de cupo máximo, unicidad de atención simultánea y concordancia de especialidades operan con absoluta precisión conforme a las especificaciones planteadas.

## 5. Conclusiones

El desarrollo de esta primera etapa demuestra que la aplicación rigurosa de los principios de encapsulamiento y separación de responsabilidades resulta esencial para gobernar sistemas complejos con múltiples reglas de negocio concurrentes, al delegar la validación de cupos y concordancia de especialidades en métodos coordinadores bien definidos, se eliminó la posibilidad de estados corruptos o asignaciones no autorizadas, preservando en todo momento la integridad referencial entre médicos, enfermeros y pacientes.

La incorporación de sobrecarga de métodos probó ser una solución arquitectónica elegante y funcionalmente superior a la implementación de métodos con identificadores repetitivos, permitir que el sistema resuelva en tiempo de compilación el destino de una asignación según la presencia de prescripciones clínicas simplifica la clase central y clarifica la intención semántica del flujo hospitalario.

Finalmente, la modelación explícita de los estados clínicos mediante tipos enumerados y el ordenamiento sustentado en el marco de colecciones de Java garantizaron un control de la evolución del paciente, con esta base estructurada proporciona un cimiento sólido para la futura incorporación de interfaces gráficas, así como las diversas implementaciones en las etapas subsecuentes del proyecto.

## Referencias

- [1] J. Gosling, B. Joy, G. Steele, G. Bracha, A. Buckley, D. Smith y G. Bierman, *The Java Language Specification, Java SE 21 Edition*, Boston, MA, USA: Addison-Wesley, 2023.
- [2] B. Eckel, *Thinking in Java*, 4th ed., Upper Saddle River, NJ, USA: Prentice Hall, 2006.
- [3] P. Deitel y H. Deitel, *Java Cómo Programar*, 10.<sup>a</sup> ed., México: Pearson Educación, 2016.
- [4] E. Gamma, R. Helm, R. Johnson y J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*, Reading, MA, USA: Addison-Wesley, 1994.
- [5] J. Bloch, *Effective Java*, 3rd ed., Boston, MA, USA: Addison-Wesley, 2018.